



JOHN THE RIPPER

The Ultimate Cybersecurity eBook

Everything You Need to Know — Step by Step

Written by

Tanish Sareen

Cybersecurity Practitioner & Ethical Hacker

⚠ FOR EDUCATIONAL AND AUTHORIZED PENETRATION TESTING USE ONLY ⚠

⚠ WARNING

LEGAL DISCLAIMER: This eBook is written solely for educational purposes and authorized penetration testing. Using John the Ripper or any password-cracking tool on systems, accounts, or networks without explicit written authorization is ILLEGAL and may result in criminal prosecution under computer fraud laws including the Computer Fraud and Abuse Act (CFAA), the UK Computer Misuse Act, and equivalent international laws. The author and publisher accept NO responsibility for misuse of the information in this book. Always obtain written permission before testing any system.

Table of Contents

Chapter 1 — Introduction to Password Cracking & John the Ripper

Chapter 2 — How John the Ripper Works (Deep Dive)

Chapter 3 — Installation on Every Platform

Chapter 4 — Core Concepts & Terminology

Chapter 5 — Your First Crack: Getting Started

Chapter 6 — Attack Modes (Complete Guide)

Chapter 7 — Working with Hash Types

Chapter 8 — Wordlists & Custom Rules

Chapter 9 — Advanced Techniques & Options

Chapter 10 — Real-World Use Cases & Walkthroughs

Chapter 11 — John the Ripper vs Hashcat

Chapter 12 — Defending Against Password Cracking

Chapter 13 — Tips, Tricks & Cheat Sheet

CHAPTER 1 INTRODUCTION TO PASSWORD CRACKING & JOHN THE RIPPER

1.1 What is Password Cracking?

Password cracking is the process of recovering passwords from data that has been stored in or transmitted by a computer system. In modern systems, passwords are rarely stored in plain text. Instead, they are processed through a one-way mathematical function called a hash, which converts a password string into a fixed-length string of characters.

When an attacker (or a penetration tester) wants to discover a password, they need to find the original input that produced a given hash output. This is the fundamental goal of password cracking.

1.2 What is John the Ripper?

John the Ripper (JtR) is one of the most widely used, open-source password security auditing and password recovery tools in the world. Originally developed by Solar Designer (Alexander Peslyak) in 1996, it has been actively maintained and improved for over 25 years.

John the Ripper is named after the infamous Victorian-era serial killer "Jack the Ripper" — the name conveys power, precision, and an ability to tear through security. It is part of the penetration tester's essential toolkit and is included in virtually every major security-focused Linux distribution, including Kali Linux and Parrot OS.

1.3 Why John the Ripper?

- ▶ Free and open-source — no licensing costs
- ▶ Supports hundreds of hash types natively
- ▶ Highly configurable with custom rules
- ▶ Cross-platform: Linux, macOS, Windows
- ▶ Community edition + "Jumbo" edition with extended support
- ▶ Excellent documentation and community support
- ▶ Can crack /etc/shadow, ZIP passwords, Office files, and more

1.4 Versions of John the Ripper

There are two main builds you will encounter:

- ▶ John the Ripper Core (Classic) — the original, maintained build from openwall.com
- ▶ John the Ripper Jumbo (Community Enhanced) — extends the core with hundreds of additional hash types, external tools integration, and formats like KeePass, Bitcoin wallets, PDF, WPA/WPA2, and more. This is the recommended version for most use cases.

1.5 Legal and Ethical Use

Before using John the Ripper in any environment, you must ensure you have explicit authorization. Here are the authorized use cases:

1. Auditing password strength on systems you own or administer
2. Penetration testing engagements with a signed scope-of-work agreement
3. Capture The Flag (CTF) competitions
4. Cybersecurity research in isolated lab environments
5. Recovering your own lost passwords

WARNING

NEVER use John the Ripper on any system, account, or hash file that you do not own or have written authorization to test. Unauthorized use is a criminal offense in most countries.

CHAPTER 2 HOW JOHN THE RIPPER WORKS

2.1 The Hashing Concept

A hash function takes an input (your password) and produces a fixed-size output called a hash or digest. Hash functions are designed to be one-way — you cannot reverse-engineer the password from the hash. Common hash algorithms include:

- ▶ MD5 — produces a 128-bit hash (32 hex characters). Fast but broken for security.
- ▶ SHA-1 — produces a 160-bit hash. Deprecated for security use.
- ▶ SHA-256 / SHA-512 — part of the SHA-2 family. Widely used today.
- ▶ bcrypt — designed specifically for passwords, intentionally slow.
- ▶ NTLM — Microsoft's hash format used in Windows authentication.
- ▶ LM (LAN Manager) — legacy Windows hash, extremely weak.
- ▶ sha512crypt — the format used in modern Linux `/etc/shadow` files.

2.2 Why Hashing Doesn't Always Protect You

Hashing alone has several weaknesses that John the Ripper exploits:

- ▶ Determinism — the same password always produces the same hash. If two users have 'password123', their hashes are identical, which reveals patterns.
- ▶ No salt (in old systems) — without a random salt value, pre-computed tables (Rainbow Tables) can instantly crack common passwords.
- ▶ Computational speed — fast hashes like MD5 can be computed billions of times per second on modern GPUs, making brute-force feasible for short passwords.
- ▶ Weak passwords — humans tend to choose predictable, dictionary-based passwords.

2.3 John's Attack Pipeline

John the Ripper works through the following pipeline for each password candidate:

6. Read the hash file and identify (or detect) the hash type
7. Generate a candidate password based on the current attack mode
8. Apply the same hash algorithm to that candidate
9. Compare the resulting hash to the stored hash
10. If they match, the password is found and stored in `john.pot`
11. If not, move to the next candidate and repeat

2.4 The John.pot File

When John cracks a password, it writes the result to a file called `john.pot` (by default located in `~/.john/john.pot`). This prevents JtR from re-cracking already-cracked passwords in future runs. You can view cracked passwords at any time using:

```
john --show hashfile.txt
```

2.5 Auto-Detection of Hash Types

One of John's most powerful features is automatic hash type detection. When you run john on a hash file, it attempts to identify the hash format by analysing the structure, length, and encoding of the hash string. However, auto-detection is not always perfect, and manually specifying the format with `--format=` is recommended for accuracy.

CHAPTER 3 INSTALLATION ON EVERY PLATFORM

3.1 Installing on Kali Linux / Parrot OS

John the Ripper is pre-installed on Kali Linux and Parrot OS. To verify and update:

```
# Check if John is installed
which john
john --version

# Update to latest version
sudo apt update && sudo apt upgrade john
```

3.2 Installing on Ubuntu / Debian

```
# Install from package manager (basic version)
sudo apt update
sudo apt install john

# Verify installation
john --version
```

For the Jumbo community version on Ubuntu:

```
# Install dependencies
sudo apt install build-essential libssl-dev libgmp-dev
sudo apt install git zlib1g-dev

# Clone the Jumbo repository
git clone https://github.com/openwall/john -b bleeding-jumbo john-jumbo
cd john-jumbo/src

# Configure and compile
./configure && make -s clean && make -sj4

# Run from the run directory
cd ../run
./john --version
```

3.3 Installing on macOS

```
# Using Homebrew (recommended)
brew install john-jumbo

# Verify
john --version
```



```
# Alternative: compile from source
git clone https://github.com/openwall/john -b bleeding-jumbo
cd john/src
./configure && make -sj4
```

3.4 Installing on Windows

For Windows, John the Ripper provides pre-built binaries:

12. Visit <https://www.openwall.com/john/> and download the Windows binary
13. Extract the ZIP archive to a directory (e.g., C:\john)
14. Open Command Prompt or PowerShell and navigate to the run folder:

```
cd C:\john\run
john.exe --version
```

Alternatively, use John through Windows Subsystem for Linux (WSL2):

```
# In WSL2 Ubuntu terminal
sudo apt install john
john --version
```

3.5 Verifying the Installation

```
# Check version and build info
john --version

# List all supported formats
john --list=formats

# List formats with a grep filter
john --list=formats | grep -i md5
```

❏ PRO TIP

The Jumbo version supports 400+ hash formats. Always prefer installing the Jumbo build for maximum compatibility and format support in real-world scenarios.

CHAPTER 4 CORE CONCEPTS & TERMINOLOGY

4.1 Key Terms You Must Know

- ▶ Hash — The fixed-length output produced by a one-way mathematical function applied to a password.
- ▶ Salt — A random value added to a password before hashing to prevent rainbow table attacks and ensure two identical passwords have different hashes.
- ▶ Plaintext — The original, human-readable password (e.g., 'password123').
- ▶ Ciphertext — In context of hashing, the hash value itself.
- ▶ Wordlist — A file containing a list of candidate passwords, one per line.
- ▶ Rainbow Table — A precomputed table of hash-to-password mappings. Effective against unsalted hashes.
- ▶ John.pot — The file where John stores cracked passwords.
- ▶ Mangling Rules — Transformation rules applied to wordlist entries to generate variations.
- ▶ Session — A named run of John, allowing you to pause and resume long cracking jobs.
- ▶ Format — The hash type being cracked (e.g., md5, sha512crypt, ntlm).
- ▶ Crack speed (c/s) — Candidates per second; how fast John is testing passwords.

4.2 The /etc/shadow File

On Linux systems, the /etc/shadow file stores hashed user passwords. Each line follows this format:

```
username:$id$salt$hash:lastchg:min:max:warn:inactive:expire:

# Examples:
root:$6$rounds=5000$salt$hash:18000:0:99999:7:::
tanish:$y$j9T$SaltValue$HashValue:19000:0:99999:7:::

# The $id$ prefix indicates the hash type:
# $1$ = MD5-crypt
# $2$ = bcrypt
# $5$ = SHA-256-crypt
# $6$ = SHA-512-crypt
# $y$ = yescrypt (modern default)
```

4.3 Understanding Hash Formats

Hash format identifiers used by John the Ripper:

```
# Common format identifiers
raw-md5          # Plain MD5 hashes
raw-sha1         # Plain SHA-1 hashes
```

```
raw-sha256      # Plain SHA-256 hashes
raw-sha512      # Plain SHA-512 hashes
md5crypt        # $1$ Linux MD5
sha256crypt     # $5$ Linux SHA-256
sha512crypt     # $6$ Linux SHA-512
bcrypt         # $2a$ bcrypt
nt              # NTLM (Windows)
lm              # LM (Legacy Windows)
descrypt        # Traditional Unix DES
zip             # ZIP archive passwords
office          # MS Office documents
pdf            # PDF passwords
wpa             # WPA/WPA2 handshakes
```

CHAPTER 5 YOUR FIRST CRACK — GETTING STARTED

5.1 Creating a Test Hash File

Before cracking anything, let's create a safe test environment with known passwords:

```
# Create an MD5 hash for 'password123'
echo -n 'password123' | md5sum
# Output: 482c811da5d5b4bc6d497ffa98491e38

# Create a test hash file
echo '482c811da5d5b4bc6d497ffa98491e38' > test_hashes.txt

# Add another hash ('hello')
echo -n 'hello' | md5sum >> test_hashes.txt

# View the file
cat test_hashes.txt
```

5.2 Your First John Command

```
# Run John against the test file (auto-detect mode)
john test_hashes.txt

# Specify the format explicitly (recommended)
john --format=raw-md5 test_hashes.txt
```

John will display output similar to:

```
Using default input encoding: UTF-8
Loaded 2 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, 'h' for help...
hello (hash2)
Proceeding with wordlist:/usr/share/john/password.lst
password123 (hash1)
2 password hashes cracked, 0 left
```

5.3 Viewing Cracked Passwords

```
# Show all cracked passwords
john --show test_hashes.txt

# Show with specific format
john --show --format=raw-md5 test_hashes.txt
```

```
# View the john.pot file directly
cat ~/.john/john.pot
```

5.4 Working with /etc/shadow (Linux)

In a real authorized penetration test on a Linux system:

```
# Step 1: Unshadow — combine /etc/passwd and /etc/shadow
unshadow /etc/passwd /etc/shadow > unshadowed.txt

# Step 2: Run John against the combined file
john unshadowed.txt

# Step 3: View results
john --show unshadowed.txt
```

WARNING

You need root/sudo access to read /etc/shadow. Only do this on systems you own or have explicit authorization to audit.

CHAPTER 6 ATTACK MODES — COMPLETE GUIDE

6.1 Overview of Attack Modes

John the Ripper has four primary attack modes, each suited to different scenarios. Understanding when to use each mode is the mark of an experienced password auditor.

6.2 Mode 1: Single Crack Mode

Single crack mode is John's fastest and most intelligent mode. It uses information derived from the username, GECOS fields (full name, room number, etc.), and home directory to generate password candidates. This mode leverages the common human behaviour of using personal information in passwords.

```
# Basic single crack mode
john --single unshadowed.txt

# With specific format
john --single --format=sha512crypt unshadowed.txt

# Single mode with custom rules
john --single --rules:Single unshadowed.txt
```

Example: If the username is 'tanish', single mode will try variations like: Tanish, TANISH, tanish1, t4n1sh, n1hsat (reversed), etc.

❏ PRO TIP

Always run Single Crack Mode FIRST. It is the fastest mode and often cracks 20-30% of passwords in poorly configured systems within seconds.

6.3 Mode 2: Wordlist Mode

Wordlist mode (also called Dictionary Attack) is the most commonly used and effective mode. John reads candidate passwords from a wordlist file and tests each one, optionally applying mangling rules to generate variations.

```
# Basic wordlist attack
john --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt

# With mangling rules
john --wordlist=rockyou.txt --rules hashes.txt

# With a specific ruleset
john --wordlist=rockyou.txt --rules:Jumbo hashes.txt

# With format specified
john --wordlist=rockyou.txt --format=raw-md5 --rules hashes.txt
```

```
# Pipe a wordlist from stdin
cat wordlist.txt | john --stdin hashes.txt
```

6.4 Mode 3: Incremental Mode (Brute Force)

Incremental mode is John's built-in brute-force mode. It generates all possible character combinations up to a specified length. This mode will eventually crack any password given enough time and computing power, but can be extremely slow for long or complex passwords.

```
# Default incremental (all printable ASCII)
john --incremental hashes.txt

# Digits only (great for PINs)
john --incremental:Digits hashes.txt

# Alpha characters only
john --incremental:Alpha hashes.txt

# Alphanumeric
john --incremental:Alnum hashes.txt

# Limit minimum and maximum password length
john --incremental --min-length=4 --max-length=8 hashes.txt

# Built-in incremental modes:
# ASCII    — all 95 printable ASCII characters
# Alpha    — 26 lowercase letters
# Digits    — 0-9
# Alnum     — letters + digits
# LowerNum  — lowercase + digits
```

6.5 Mode 4: External Mode

External mode allows you to write custom C-like code in John's configuration file to implement custom password generation logic. This is the most flexible but most complex mode, requiring programming knowledge.

```
# Use a predefined external mode
john --external:Double hashes.txt

# Use a custom external mode defined in john.conf
john --external:MyCustomMode hashes.txt
```

External modes are defined in `/etc/john/john.conf` (or `john.conf` in the John directory) under `[List.External:ModeName]` sections. They define functions like `init()`, `generate()`, `filter()`, and `restore()` using a C-like syntax.

6.6 Combining Modes — Recommended Strategy

An experienced penetration tester follows this attack order for maximum efficiency:

- 15.** Single mode — fast, uses account metadata
- 16.** Wordlist with rules (rockyou.txt + Jumbo rules)
- 17.** Wordlist with rules (custom/targeted wordlists)
- 18.** Incremental for short passwords (max-length=6)
- 19.** Incremental for longer passwords (overnight/long-running)

CHAPTER 7 WORKING WITH HASH TYPES

7.1 Identifying Hash Types

Before cracking, you need to identify what type of hash you have. There are several ways to do this:

```
# Method 1: Let John auto-detect
john unknown_hash.txt

# Method 2: Use hash-identifier tool
hash-identifier
# Then paste your hash

# Method 3: Use hashid
pip install hashid
hashid '5f4dcc3b5aa765d61d8327deb882cf99'

# Method 4: Examine hash length and prefix
# 32 chars hex = MD5
# 40 chars hex = SHA-1
# 64 chars hex = SHA-256
# $6$ prefix   = sha512crypt
# $2a$ prefix  = bcrypt
```

7.2 Cracking MD5 Hashes

```
# Create test MD5 hash
echo -n 'tanish123' | md5sum | cut -d' ' -f1 > md5_hash.txt

# Crack it
john --format=raw-md5 --wordlist=rockyou.txt md5_hash.txt

# Show result
john --show --format=raw-md5 md5_hash.txt
```

7.3 Cracking SHA-1 Hashes

```
# Create SHA-1 hash
echo -n 'mypassword' | shasum | cut -d' ' -f1 > sha1_hash.txt

# Crack with wordlist
john --format=raw-sha1 --wordlist=rockyou.txt sha1_hash.txt
```

7.4 Cracking SHA-512 Hashes

```
# sha512crypt is the format used in modern Linux /etc/shadow
# Example hash from /etc/shadow
echo 'user:$6$rounds=5000$randomsalt$HashHere:18900:0:99999:7:::' > sha512.txt

john --format=sha512crypt --wordlist=rockyou.txt sha512.txt
```

7.5 Cracking Windows NTLM Hashes

NTLM is Windows' primary password hashing mechanism. You can extract NTLM hashes from Windows using tools like Mimikatz or from the SAM database.

```
# NTLM hash format
# username:RID:LMhash:NThash:::
echo
'tanish:1001:aad3b435b51404eeaad3b435b51404ee:8846f7eaae8fb117ad06bdd830b7586c:::'
> ntlm.txt

# Crack NTLM
john --format=nt ntlm.txt

# Or with wordlist
john --format=nt --wordlist=rockyou.txt ntlm.txt

# Show results
john --show --format=nt ntlm.txt
```

7.6 Cracking bcrypt Hashes

bcrypt is intentionally slow, making it resistant to brute-force attacks. Cracking bcrypt hashes is much slower than other formats.

```
# bcrypt hash example
echo '$2a$12$SaltValueHere$HashValueHere' > bcrypt.txt

# Crack with wordlist (will be slow due to bcrypt design)
john --format=bcrypt --wordlist=rockyou.txt bcrypt.txt

# Monitor progress
john --status
```

❏ PRO TIP

bcrypt with cost factor 12+ can take hours to crack even a simple password. This is by design — bcrypt is one of the strongest password hashing algorithms available.

7.7 Cracking ZIP File Passwords

```
# Step 1: Extract the hash from the ZIP file
zip2john protected.zip > zip_hash.txt

# Step 2: View the extracted hash
cat zip_hash.txt

# Step 3: Crack with wordlist
john --wordlist=rockyou.txt zip_hash.txt

# Step 4: Show the cracked password
john --show zip_hash.txt
```

7.8 Cracking PDF Passwords

```
# Extract hash from PDF
pdf2john.pl document.pdf > pdf_hash.txt

# Crack with wordlist
john --format=PDF --wordlist=rockyou.txt pdf_hash.txt

# Show result
john --show pdf_hash.txt
```

7.9 Cracking Microsoft Office Passwords

```
# Word document
office2john.py document.docx > office_hash.txt

# Excel spreadsheet
office2john.py spreadsheet.xlsx >> office_hash.txt

# Crack
john --wordlist=rockyou.txt office_hash.txt
```

7.10 Cracking SSH Private Key Passphrases

```
# Extract hash from SSH key
ssh2john.py id_rsa > ssh_hash.txt

# Crack passphrase
john --wordlist=rockyou.txt ssh_hash.txt

# Show result
john --show ssh_hash.txt
```

CHAPTER 8 WORDLISTS & CUSTOM RULES

8.1 Popular Wordlists

The quality of your wordlist is critical to cracking success. Here are the most widely used:

- ▶ rockyou.txt — 14 million+ passwords from the 2009 RockYou breach. Available at `/usr/share/wordlists/rockyou.txt` on Kali Linux.
- ▶ SecLists — A comprehensive collection of wordlists for security testing: github.com/danielmiessler/SecLists
- ▶ CrackStation — 1.5 billion word dictionary: crackstation.net/crackstation-wordlist-password-cracking-dictionary.htm
- ▶ Probable-Wordlists — Sorted by probability of occurrence
- ▶ Custom — Wordlists generated from target OSINT (best for targeted attacks)

8.2 Preparing Wordlists

```
# Decompress rockyou.txt if gzipped
gunzip /usr/share/wordlists/rockyou.txt.gz

# Count entries in wordlist
wc -l /usr/share/wordlists/rockyou.txt

# Sort and deduplicate a wordlist
sort -u wordlist.txt > sorted_wordlist.txt

# Combine multiple wordlists
cat wordlist1.txt wordlist2.txt wordlist3.txt | sort -u > combined.txt
```

8.3 Creating Custom Wordlists with CUPP

CUPP (Common User Passwords Profiler) generates targeted wordlists based on OSINT about a target individual. Extremely effective for targeted attacks with authorization.

```
# Install CUPP
git clone https://github.com/Mebus/cupp.git
cd cupp && python3 cupp.py -i

# CUPP will interactively ask for:
# - Target's name, surname, nickname
# - Birthday, partner, children
# - Pet names, company names
# It then generates a targeted wordlist
```

8.4 Understanding John's Rule Syntax

Rules are transformations applied to wordlist entries. They are defined in `john.conf` and can be specified with `--rules`. Common rule operations include:

- ▶ `l` — Convert to lowercase
- ▶ `u` — Convert to uppercase
- ▶ `c` — Capitalize first letter
- ▶ `r` — Reverse the word
- ▶ `d` — Duplicate the word
- ▶ `$X` — Append character `X` to end
- ▶ `^X` — Prepend character `X` to front
- ▶ `sXY` — Replace all `X` with `Y`
- ▶ `[N]` — Truncate to `N` characters
- ▶ `Az"[0-9]"` — Append a digit

8.5 Using Built-in Rule Sets

```
# No rules (pure wordlist)
john --wordlist=rockyou.txt hashes.txt

# Default rules (moderate transformations)
john --wordlist=rockyou.txt --rules hashes.txt

# Single rules (used with single mode)
john --wordlist=rockyou.txt --rules:Single hashes.txt

# Wordlist rules
john --wordlist=rockyou.txt --rules:Wordlist hashes.txt

# Jumbo rules (thousands of transformations, very thorough)
john --wordlist=rockyou.txt --rules:Jumbo hashes.txt

# KoreLogic rules (competition-grade, very comprehensive)
john --wordlist=rockyou.txt --rules:KoreLogic hashes.txt
```

8.6 Writing Custom Rules

You can add custom rules to `john.conf`. Here's how to write your own:

```
# In john.conf, under [List.Rules:MyCustomRules]

[List.Rules:MyCustomRules]
# Original word
:
# Uppercase
u
# Lowercase
l
```

```
# Capitalize
c
# Append 1, 2, 3
$1
$2
$3
# Append 123
$1$2$3
# Prepend @ symbol
^@
# Replace a with @
sa@
# Replace e with 3
se3
# Reverse and capitalize
rc

# Use your custom rules
john --wordlist=rockyou.txt --rules:MyCustomRules hashes.txt
```

CHAPTER 9 ADVANCED TECHNIQUES & OPTIONS

9.1 Session Management

For long cracking jobs, session management allows you to pause and resume without losing progress.

```
# Start a named session
john --session=mysession --wordlist=rockyou.txt hashes.txt

# Restore a paused session
john --restore=mysession

# List all active sessions
john --list=sessions

# Status of a running session (from another terminal)
john --status=mysession

# Press 'S' during a run to see current status
# Press 'Q' or Ctrl+C to pause (saves state automatically)
```

9.2 Monitoring Progress

```
# Check status of a running job
john --status

# From another terminal while John is running
kill -USR1 $(pidof john)

# Sample output:
# Session.inProgress.....: yes
# Loaded hashes.....: 100
# Remaining hashes.....: 95
# Guesses: 5   Time: 0:00:30   c/s: 125000   trying: tanish2 - tanish9
```

9.3 Multi-CPU / Multi-Threading

```
# Use OpenMP to utilize multiple CPU cores
OMP_NUM_THREADS=8 john --wordlist=rockyou.txt hashes.txt

# Or with --fork for multiple processes
john --fork=4 --wordlist=rockyou.txt hashes.txt

# Check available CPU cores
nproc
```

```
# Use all available cores
john --fork=$(nproc) --wordlist=rockyou.txt hashes.txt
```

9.4 Loopback Mode

Loopback mode cracks hashes using previously cracked passwords as a wordlist. This is powerful because cracked passwords often share patterns with other passwords in the same dataset.

```
# Use the pot file as a wordlist
john --loopback hashes.txt

# Loopback with rules
john --loopback --rules hashes.txt

# Use a specific pot file
john --loopback --pot=mycracked.pot hashes.txt
```

9.5 Mask Mode (John Jumbo)

Mask mode is similar to incremental but allows you to specify patterns (masks) for password generation. This is ideal when you know part of the password structure.

```
# Mask syntax: ?l=lowercase, ?u=uppercase, ?d=digit, ?s=special, ?a=all

# 8 lowercase letters
john --mask='?l?l?l?l?l?l?l?l' hashes.txt

# Capital letter + 5 lowercase + 2 digits
john --mask='?u?l?l?l?l?l?d?d' hashes.txt

# Known prefix (know password starts with 'tan')
john --mask='tan?l?l?l?d?d' hashes.txt

# Custom charset: define ?l as specific characters
john --mask='?l?l?l?l?d?d' -l='aeiou' hashes.txt
```

9.6 Pipe Mode (stdin)

```
# Pipe candidates directly from command line tools
# Example: generate candidates with Python
python3 -c "
for i in range(1000, 9999):
    print(f'tanish{i}')
" | john --stdin hashes.txt

# Use crunch to generate candidates and pipe to John
crunch 6 8 abcdefghijklmnopqrstuvwxyz0123456789 | john --stdin hashes.txt
```



```
# Use Mentalist or other tools
cat generated_words.txt | john --stdin hashes.txt
```

9.7 Adjusting John's Configuration

John's behaviour can be fine-tuned through the john.conf file. Key settings include:

```
# Location of john.conf
# Linux:   /etc/john/john.conf or ~/.john/john.conf
# Windows: john\run\john.conf

# Important configuration sections:
[Incremental:Alpha]      # Character set for Alpha incremental
[Incremental:Digits]     # Character set for Digits incremental
[List.Rules:Single]      # Rules for single crack mode
[List.Rules:Wordlist]    # Rules for wordlist mode
[List.Rules:Jumbo]       # Jumbo/KoreLogic rules
[Options]                # Global options
```

CHAPTER 10 REAL-WORLD USE CASES & WALKTHROUGHS

10.1 Walkthrough: Auditing Linux System Passwords

Scenario: You are a system administrator asked to audit password strength on an Ubuntu 22.04 server you manage.

20. Extract the hashes using unshadow:

```
sudo unshadow /etc/passwd /etc/shadow > audit_hashes.txt
cat audit_hashes.txt # Verify content
```

21. Identify hash type (modern Ubuntu uses sha512crypt):

```
john --list=formats | grep sha512
```

22. Run Single mode first:

```
john --single --format=sha512crypt audit_hashes.txt
```

23. Follow with wordlist attack:

```
john --wordlist=/usr/share/wordlists/rockyou.txt --rules --format=sha512crypt
audit_hashes.txt
```

24. Check results and report weak passwords:

```
john --show --format=sha512crypt audit_hashes.txt
```

10.2 Walkthrough: Cracking a ZIP File

Scenario: You are a forensic analyst recovering evidence from a password-protected ZIP file (authorized investigation).

25. Extract the hash from the ZIP:

```
zip2john evidence.zip > zip_hash.txt
cat zip_hash.txt
```

26. Try wordlist attack first:

```
john --wordlist=rockyou.txt zip_hash.txt
```

27. If not found, try with rules:

```
john --wordlist=rockyou.txt --rules:Jumbo zip_hash.txt
```

28. If still not found, try incremental for short passwords:

```
john --incremental --max-length=6 zip_hash.txt
```

29. Display the cracked password:

```
john --show zip_hash.txt
```

10.3 Walkthrough: CTF Challenge — Multi-Hash File

Scenario: A Capture the Flag competition provides a file with multiple hash types.

```
# Examine the hash file
cat ctf_hashes.txt

# Let John auto-detect and crack
john ctf_hashes.txt

# If auto-detect fails, identify manually
hashid 'hash_value_here'

# Crack each format explicitly
john --format=raw-md5 ctf_hashes.txt
john --format=raw-sha1 ctf_hashes.txt

# Try wordlist with rules
john --wordlist=rockyou.txt --rules:All ctf_hashes.txt

# Show all cracked passwords
john --show ctf_hashes.txt
```

10.4 Walkthrough: WPA/WPA2 Wi-Fi Password Cracking

Scenario: You are a wireless security auditor testing your own network's WPA2 passphrase strength.

```
# Step 1: Capture WPA handshake using aircrack-ng (on your own network)
sudo airmon-ng start wlan0
sudo airodump-ng -c 6 --bssid XX:XX:XX:XX:XX:XX -w capture wlan0mon

# Step 2: Convert capture to John format (using hcxtools)
hcxpcapngtool -o hash.hc22000 capture.pcapng

# OR use cap2john
cap2john capture.pcap > wpa_hash.txt

# Step 3: Crack with wordlist
john --format=wpapsk wpa_hash.txt --wordlist=rockyou.txt
```

```
# Step 4: Show result
john --show --format=wpapsk wpa_hash.txt
```

⚠ WARNING

Only test Wi-Fi networks you own or have explicit written permission to test. Unauthorized wireless testing is illegal under computer misuse laws worldwide.

CHAPTER 11 JOHN THE RIPPER vs HASHCAT

11.1 Overview

Both John the Ripper and Hashcat are industry-standard password cracking tools. Each has strengths that make it better suited for specific scenarios. Understanding both tools makes you a more effective security professional.

11.2 Comparison Table

Feature	John the Ripper	Hashcat
GPU Support	Limited (CPU-focused)	Excellent (CUDA/OpenCL)
Speed (fast hashes)	Moderate	Extremely fast
Speed (slow hashes)	Good	Good
Ease of Use	Very Easy	Moderate
Hash Format Support	400+ (Jumbo)	300+
File Cracking (ZIP/PDF/Office)	Excellent (built-in)	Requires tools
Auto-detect Hash Type	Yes	Manual
Rule Language	Own syntax	Hashcat rules
Default Mode	Auto (smart)	Manual required
Best For	Linux audits, files, CTFs	Large hash sets, GPU farms

11.3 When to Use Which

- ▶ Use John the Ripper when: auditing Linux/Unix systems, cracking file passwords (ZIP, PDF, Office, SSH keys), working in CTF environments, or when you need auto hash detection.
- ▶ Use Hashcat when: you have a GPU available, working with large hash dumps (millions of hashes), need maximum speed, or cracking NTLM hashes from Windows environments at scale.

❏ PRO TIP

Many professionals use both tools together: use John for initial triage and file cracking, then pass remaining hashes to Hashcat for GPU-accelerated brute force.

CHAPTER 12 DEFENDING AGAINST PASSWORD CRACKING

12.1 Why Defenders Must Understand Offense

Understanding how tools like John the Ripper work is essential for defenders. You cannot build effective password policies without knowing how attackers approach password recovery. This chapter covers defensive measures to make password cracking infeasible.

12.2 Password Hashing Best Practices

- ▶ Use bcrypt, Argon2, or scrypt — these are memory-hard algorithms designed to be slow and resource-intensive, making brute-force attacks expensive.
- ▶ Never use MD5 or SHA-1 for password storage — these are fast hash functions not designed for passwords.
- ▶ Always use a unique, random salt — this prevents rainbow table attacks and ensures identical passwords have different hashes.
- ▶ Use high work factors/cost parameters — for bcrypt, use cost factor 12 or higher on modern hardware.

12.3 Password Policy Recommendations

- ▶ Minimum 12 characters (ideally 16+)
- ▶ Require a mix of uppercase, lowercase, digits, and special characters
- ▶ Prohibit common passwords — check against HaveIBeenPwned's database
- ▶ Implement account lockout after 5-10 failed attempts
- ▶ Use multi-factor authentication (MFA) as the primary defense
- ▶ Force password changes only when compromise is suspected (not on schedule)
- ▶ Educate users to use passphrases — 'CorrectHorseBatteryStaple' is stronger than 'P@ssw0rd1'

12.4 What Makes a Password Resistant to Cracking?

```
# Weak passwords (cracked in seconds):
password, 123456, qwerty, letmein, tanish1

# Moderate passwords (cracked in hours-days):
P@ssw0rd, Tanish2024!, Summer#2024

# Strong passwords (practically uncrackable):
X#9mK@vL2pQr7bWn   <- 16-char random
correct-horse-battery-staple <- passphrase
7$Lp!QzN3@mB&vR8kJ <- 18-char mixed random
```

```
# Time to crack with 10 billion guesses/second:  
8-char lowercase only : < 1 second  
8-char mixed          : ~5 hours  
12-char mixed         : ~34,000 years  
16-char random        : > universe age
```

12.5 Monitoring for Password Attacks

- ▶ Monitor failed login attempts and trigger alerts on unusual patterns
- ▶ Use a SIEM (Security Information and Event Management) system to correlate authentication logs
- ▶ Deploy honeypot accounts with known passwords to detect credential stuffing
- ▶ Regularly audit /etc/shadow for new or modified accounts
- ▶ Use tools like fail2ban to automatically block IPs with excessive failed logins

CHAPTER 13 TIPS, TRICKS & COMPLETE CHEAT SHEET

13.1 Essential Commands Quick Reference

```
# ——— INSTALLATION ———
sudo apt install john          # Kali/Ubuntu
brew install john-jumbo       # macOS

# ——— BASIC USAGE ———
john hashes.txt               # Auto mode
john --format=FORMAT hashes.txt # Specify format
john --show hashes.txt        # Show cracked
john --show --format=FORMAT hashes.txt

# ——— ATTACK MODES ———
john --single hashes.txt      # Single crack
john --wordlist=WORDLIST hashes.txt # Dictionary
john --wordlist=WORDLIST --rules hashes.txt # Dict + rules
john --incremental hashes.txt  # Brute force
john --incremental:Digits hashes.txt # Digits only
john --mask='?u?l?l?l?d?d' hashes.txt # Mask mode

# ——— LINUX SHADOW ———
unshadow /etc/passwd /etc/shadow > out.txt
john out.txt

# ——— FILE CRACKING ———
zip2john file.zip > hash.txt && john hash.txt
pdf2john.pl file.pdf > hash.txt && john hash.txt
office2john.py file.docx > hash.txt && john hash.txt
ssh2john.py id_rsa > hash.txt && john hash.txt

# ——— SESSION MANAGEMENT ———
john --session=NAME ARGS      # Named session
john --restore=NAME           # Resume session
john --status                  # Check status

# ——— PERFORMANCE ———
john --fork=4 ARGS            # 4 processes
OMP_NUM_THREADS=8 john ARGS   # 8 CPU threads
john --loopback hashes.txt     # Use .pot file

# ——— INFORMATION ———
john --list=formats            # All formats
john --list=formats | grep -i KEYWORD # Filter formats
john --version                 # Version info

# ——— WINDOWS/NTLM ———
john --format=nt hashes.txt    # NTLM format
john --format=lm hashes.txt    # LM format
```

13.2 Pro Tips

- ▶ Always run Single mode before Wordlist — it's fastest and exploits human naming habits.
- ▶ Use `--rules:Jumbo` instead of plain `--rules` for much more thorough mangling.
- ▶ The `john.pot` file persists between runs — delete it with care if you want to re-crack hashes.
- ▶ Use `--fork=$(nproc)` to automatically use all available CPU cores.
- ▶ For large hash sets, sort the hash file and remove duplicates first to avoid redundant work.
- ▶ When cracking `bcrypt`, start with the most targeted wordlist — brute force is practically infeasible.
- ▶ Keep a curated, sorted wordlist built from your own successful cracks — every pentest adds to it.
- ▶ Use `john --list=formats | grep -i` to quickly find the right format identifier.
- ▶ In CTF competitions, always check if the hash is just an encoding (Base64, hex) rather than a hash.
- ▶ The Jumbo community build adds hundreds of additional formats — always prefer it over the core build.

13.3 Common Mistakes to Avoid

- ▶ Not specifying `--format`: Auto-detection sometimes fails silently, reporting 0 hashes loaded.
- ▶ Wrong hash format in file: Ensure hashes are in the correct format for the algorithm being cracked.
- ▶ Running incremental without length limits: Can run for days on a laptop with no results.
- ▶ Forgetting to check `john.pot`: Previously cracked passwords won't show in a new run without `--show`.
- ▶ Using a small wordlist: `rockyou.txt` is the minimum — combine multiple wordlists for better results.
- ▶ Not using rules with wordlists: Plain wordlist attacks miss common variations like `P@ssword1`.

13.4 Useful Hash Extraction Tools

```
zip2john          # ZIP archives
pdf2john.pl       # PDF files
office2john.py    # MS Office documents
ssh2john.py       # SSH private keys
keepass2john      # KeePass databases
bitlocker2john    # BitLocker volumes
truecrypt2john    # TrueCrypt volumes
pgp2john          # GPG/PGP keys
pwsafe2john       # Password Safe files
hccap2john        # WPA handshakes
wpacap2john       # WPA pcap captures
dmg2john          # macOS DMG files
```

FINAL NOTES

Written with passion for cybersecurity education by Tanish Sareen

John the Ripper is one of the most powerful tools in the security professional's arsenal. Mastering it takes practice — set up a safe lab environment with virtual machines, create your own hash files, and methodically work through each attack mode and option.

Remember that tools like John the Ripper are force multipliers for both attackers and defenders. The same knowledge that allows a red team to crack weak passwords enables a blue team to audit and strengthen password policies. Your job as a security professional is to use this knowledge responsibly.

Continue learning, stay curious, and always hack ethically. The cybersecurity field rewards those who understand both sides of the equation — offense informs defense, and vice versa.

— Tanish Sareen

Cybersecurity Practitioner | Ethical Hacker | eBook Author

□ **Knowledge is power. Use it wisely.** □